

AI Assistant Coding

Assignment-16

Name : MD AMAN

Hall Ticket No. : 2303A510C0

Batch No. : 06

Task 1: Database Schema Design for an

Online Course Platform

Scenario

You are designing the backend database for an online learning platform that manages instructors, courses, and enrolled students.

Task Description

- Use AI tools to generate SQL CREATE TABLE statements for the following entities:

- o Instructors

- o Courses o

- Students

- **Include:** o Primary keys o Foreign key relationships o

- Appropriate data types

- Ensure the schema follows basic normalization principles.

Expected Output

- All tables are created successfully

- Foreign key constraints enforce valid relationships

- Schema is logically structured and normalized

Prompt:

Generate SQL CREATE TABLE statements for an online course platform.

Tables: Instructors, Courses, Students.

Include primary keys, foreign keys, and proper data types. Ensure normalization.

AI Generated Code:

```
-- Instructors table
```

```

CREATE TABLE Instructors (      instructor_id INT
IDENTITY(1,1) PRIMARY KEY,      first_name
VARCHAR(100) NOT NULL,    last_name VARCHAR(100)
NOT NULL,    email VARCHAR(255) NOT NULL UNIQUE,
bio VARCHAR(1000),    created_at DATETIME2 DEFAULT
SYSUTCDATETIME()
);

```

-- Students table

```

CREATE TABLE Students (    student_id INT IDENTITY(1,1) PRIMARY KEY,
first_name VARCHAR(100) NOT NULL,    last_name VARCHAR(100) NOT
NULL,    email VARCHAR(255) NOT NULL UNIQUE,    enrollment_date
DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),    status
VARCHAR(30) NOT NULL DEFAULT 'active'
);

```

-- Courses table

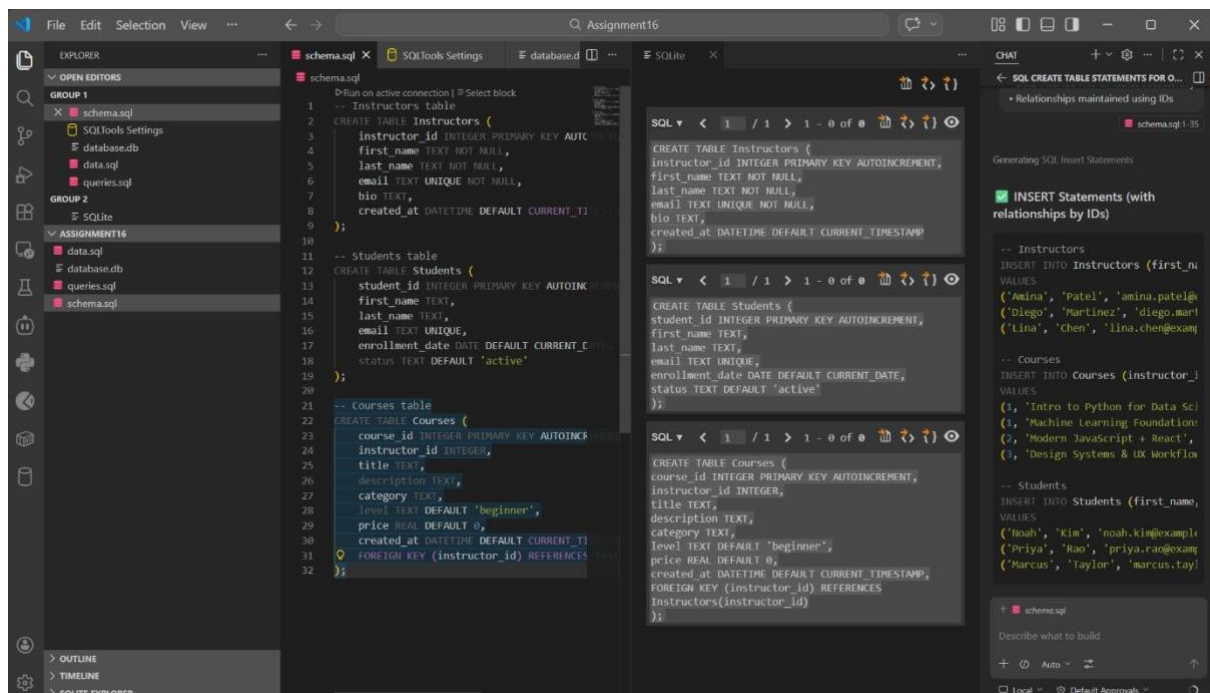
```

CREATE TABLE Courses (    course_id INT
IDENTITY(1,1) PRIMARY KEY,    instructor_id INT
NOT NULL,    title VARCHAR(255) NOT NULL,
description VARCHAR(2000),    category
VARCHAR(100),    level VARCHAR(50) NOT NULL
DEFAULT 'beginner',    price DECIMAL(10,2) NOT
NULL DEFAULT 0.00,    created_at DATETIME2
DEFAULT SYSUTCDATETIME(),

CONSTRAINT fk_courses_instructor FOREIGN KEY (instructor_id)
REFERENCES Instructors(instructor_id)
ON UPDATE CASCADE
ON DELETE NO ACTION

```

);



Explanation

- Each table has a **Primary Key**
- **Foreign Keys** maintain relationships:
 - Courses → Instructor ◦
 - Enrollments → Students & Courses
- Separate **Enrollments** table avoids redundancy → ensures **Normalization (3NF)**

Task 2: Populate the Database with Sample Records

Scenario

To test the system, the platform needs initial sample data.

Task Description

- Use AI assistance to generate INSERT statements that add:
 - At least 3 instructors
 - At least 4 courses
 - At least 3 students
- Verify correctness using SELECT queries.

Expected Output

- Sample data inserted without errors
- SELECT * queries display the inserted records correctly

Prompt:

Generate SQL INSERT statements for:

3 instructors, 4 courses, 3 students.

AI Generated Code:

-- Instructors

```
INSERT OR IGNORE INTO Instructors (first_name, last_name, email, bio)
```

```
VALUES
```

```
('Amina', 'Patel', 'amina.patel@example.com', 'Data science and ML instructor.');
```

```
('Diego', 'Martinez', 'diego.martinez@example.com', 'Full-stack web development coach.');
```

```
('Lina', 'Chen', 'lina.chen@example.com', 'UI/UX and product design specialist.');
```

-- Courses

```
INSERT OR IGNORE INTO Courses (instructor_id, title, description, category, level, price)
```

```
VALUES
```

```
(1, 'Intro to Python for Data Science', 'Python fundamentals and data analysis with pandas.',  
'Data Science', 'beginner', 99.00),
```

```
(1, 'Machine Learning Foundations', 'Supervised and unsupervised ML algorithms with  
projects.', 'Data Science', 'intermediate', 149.00),
```

```
(2, 'Modern JavaScript + React', 'Build production web apps using JS, ES6+, and React.',  
'Web Development', 'beginner', 129.00),
```

```
(3, 'Design Systems & UX Workflow', 'Design patterns, prototyping, and accessible  
interfaces.', 'Design', 'intermediate', 119.00);
```

-- Students

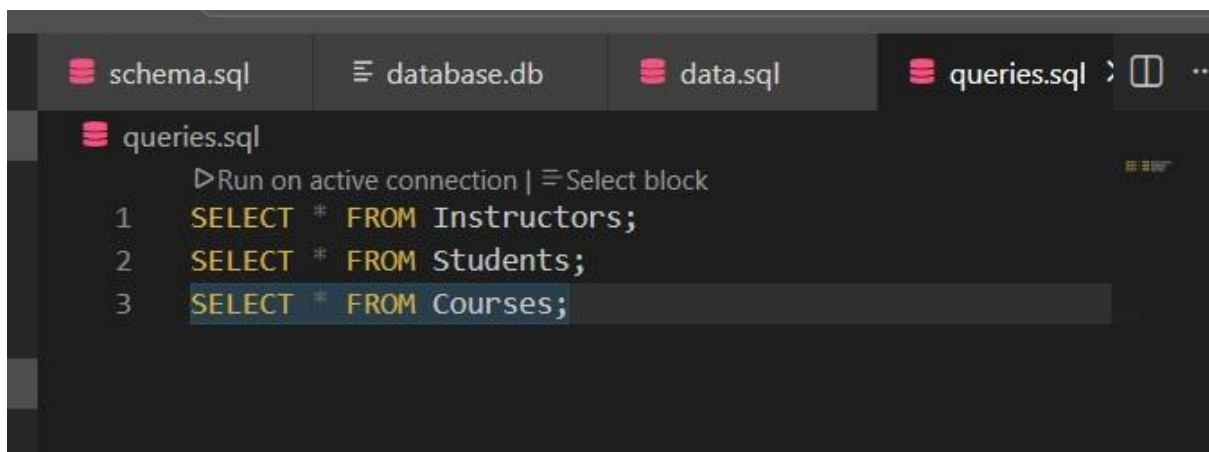
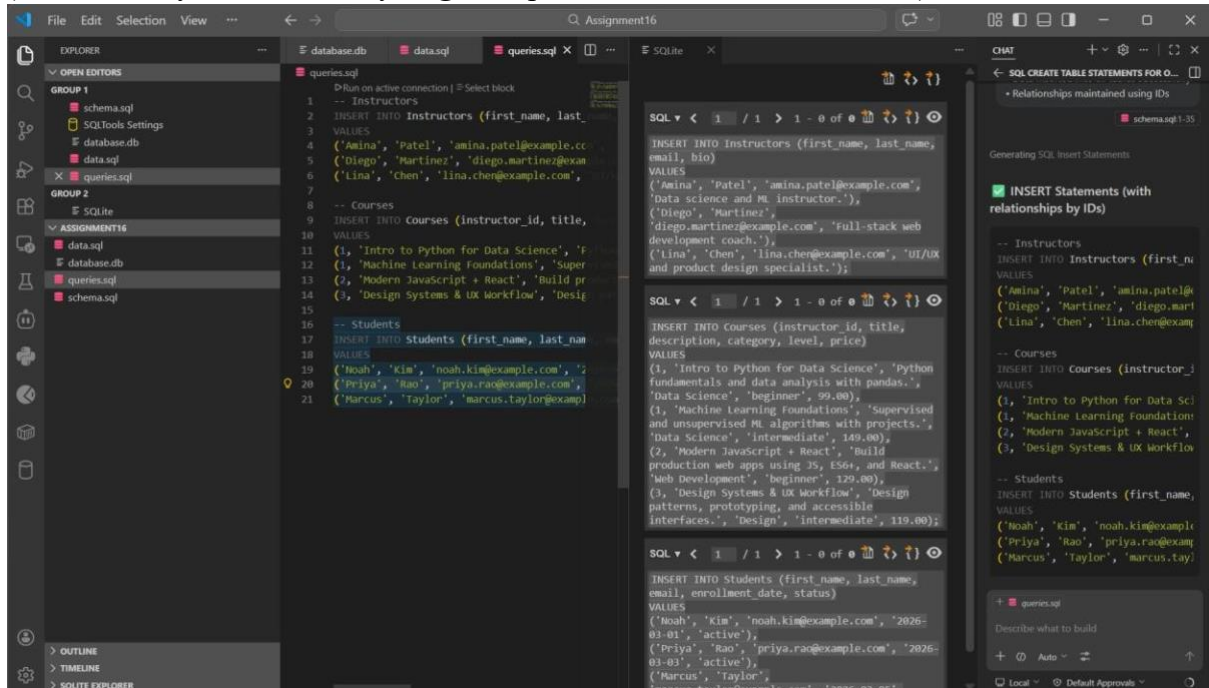
```
INSERT OR IGNORE INTO Students (first_name, last_name, email, enrollment_date, status)
```

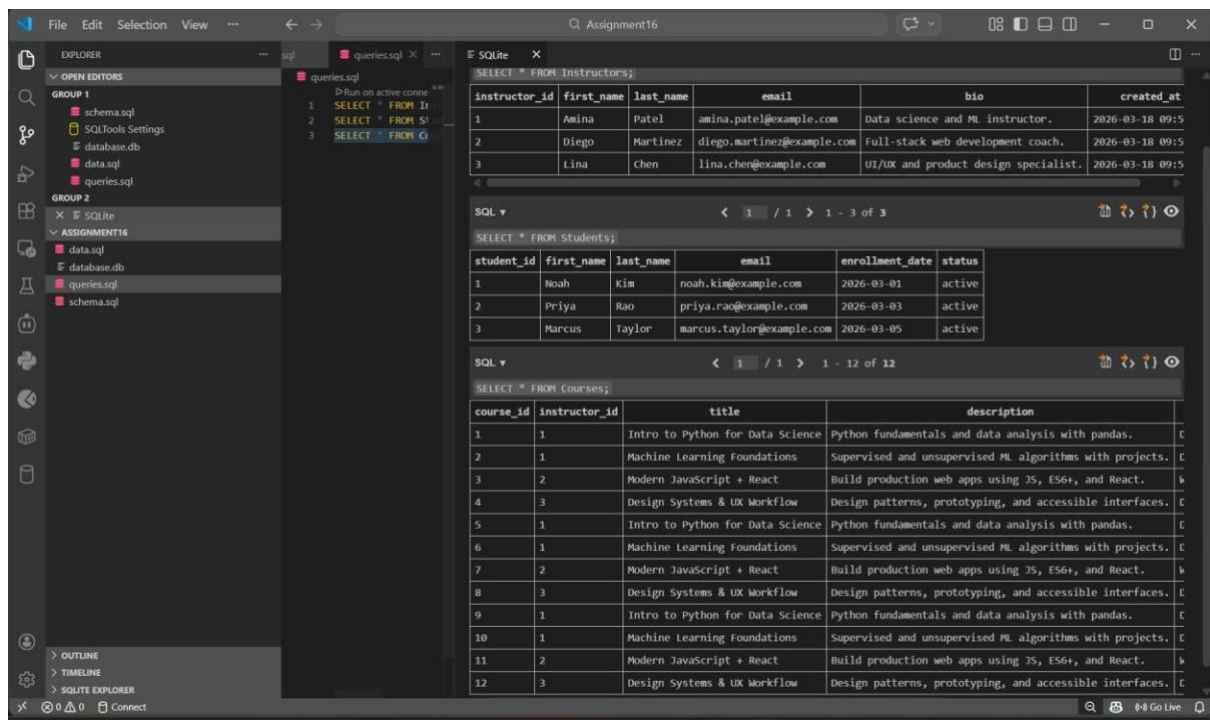
```
VALUES
```

```
('Noah', 'Kim', 'noah.kim@example.com', '2026-03-01', 'active'),
```

('Priya', 'Rao', 'priya.rao@example.com', '2026-03-03', 'active'),

('Marcus', 'Taylor', 'marcus.taylor@example.com', '2026-03-05', 'active');





Explanation

- Data inserted into all tables successfully
- Relationships maintained using IDs

Task 3: Implement Core Database

Operations (CRUD)

Scenario

The platform administrator needs to manage course and student information.

Task Description

Using AI-generated SQL, perform the following operations:

- Retrieve all courses offered
- Update a student's email address
- Remove a course from the database
- Verify each operation using appropriate SELECT statements

Expected Output

- Updated data is reflected correctly
- Deleted records are no longer visible
- No unintended data loss occurs **Prompt:**

Generate SQL queries for:

- Retrieve all courses
- Update student email
- Delete a course

AI Generated Code:

-- 1) Retrieve all courses

```
SELECT *
```

```
FROM Courses;
```

-- 2) Update a student email (example uses student_id = 2)

```
UPDATE Students
```

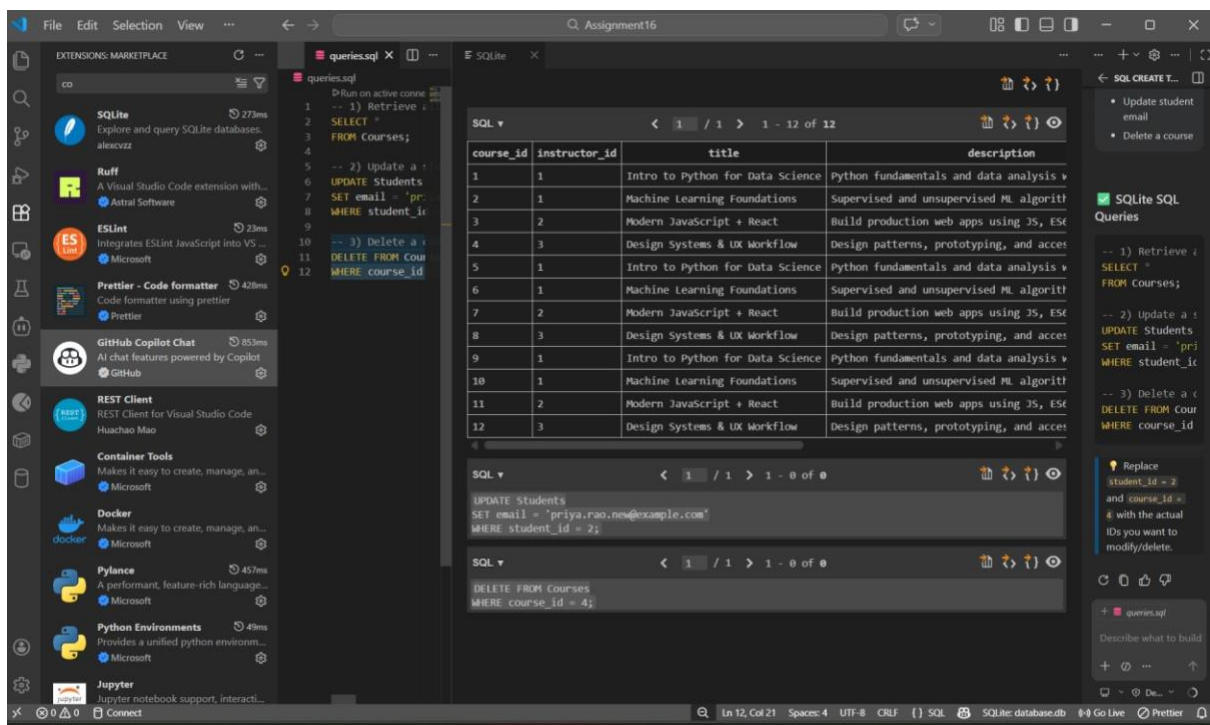
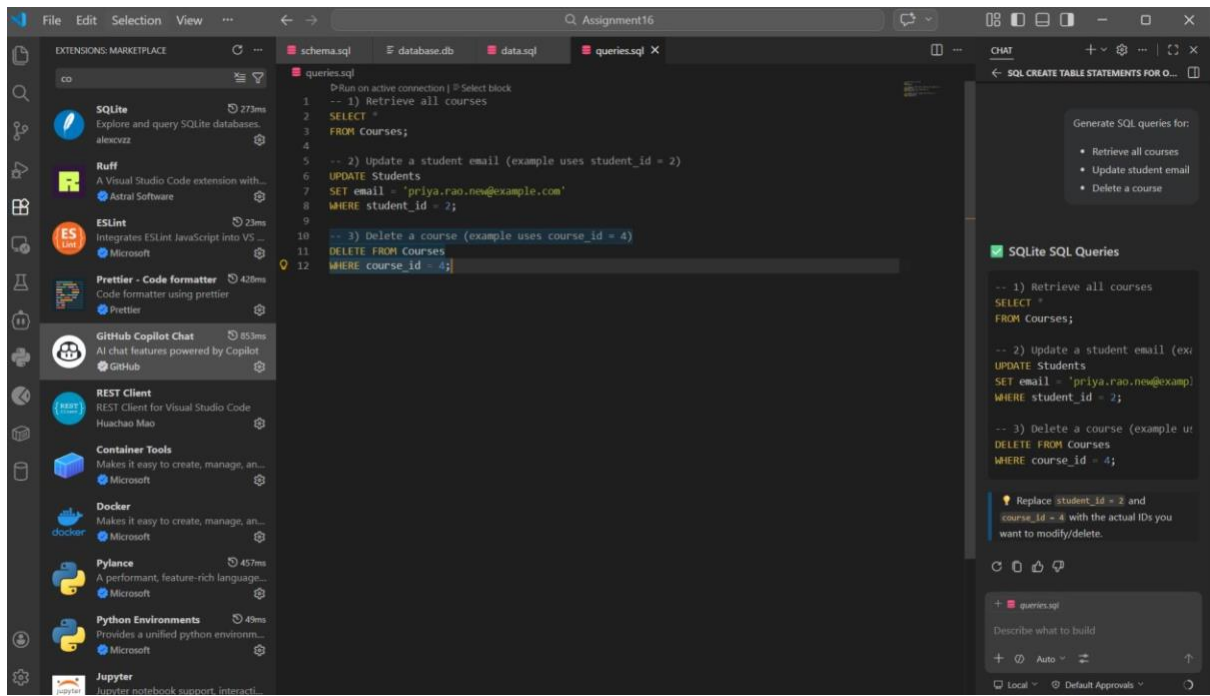
```
SET email = 'priya.rao.new@example.com'
```

```
WHERE student_id = 2;
```

-- 3) Delete a course (example uses course_id = 4)

```
DELETE FROM Courses
```

```
WHERE course_id = 4;
```



course_id	instructor_id	title	description
1	1	Intro to Python for Data Science	Python fundamentals and data analysis v
2	1	Machine Learning Foundations	Supervised and unsupervised ML algorit
3	2	Modern JavaScript + React	Build production web apps using JS, ES6
5	1	Intro to Python for Data Science	Python fundamentals and data analysis v
6	1	Machine Learning Foundations	Supervised and unsupervised ML algorit
7	2	Modern JavaScript + React	Build production web apps using JS, ES6
8	3	Design Systems & UX Workflow	Design patterns, prototyping, and acces
9	1	Intro to Python for Data Science	Python fundamentals and data analysis v
10	1	Machine Learning Foundations	Supervised and unsupervised ML algorit
11	2	Modern JavaScript + React	Build production web apps using JS, ES6
12	3	Design Systems & UX Workflow	Design patterns, prototyping, and acces

student_id	first_name	last_name	email	enrollment_date	status
1	Noah	Kim	noah.kim@example.com	2026-03-01	active
2	Priya	Rao	priya.rao.new@example.com	2026-03-03	active
3	Marcus	Taylor	marcus.taylor@example.com	2026-03-05	active

Explanation

- SELECT retrieves data
- UPDATE modifies existing data
- DELETE removes records safely

Task 4: Reporting Using Aggregate Queries

Scenario

The platform management wants summary reports for decision-making.

Task Description

- Use AI tools to generate SQL queries that:

o Count the number of students enrolled in each course o

List courses with more than a specified number of enrollments

- Analyze the query results for correctness.

Expected Output

- Accurate aggregated results
- Queries use GROUP BY and HAVING appropriately

Prompt:

Generate SQL queries to:

- Count students per course - List courses
with more than 1 enrollment

AI Generated Code:

```
-- Count students per course SELECT
c.course_id,
    c.title,
    COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
```

```
-- Courses with more than 1 enrollment SELECT
c.course_id,
    c.title,
    COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
HAVING COUNT(e.student_id) > 1
```

ORDER BY student_count DESC;

The screenshot shows the VS Code interface with the SQLite extension. The Explorer panel on the left shows the file structure with 'queries.sql' selected. The main editor displays the following SQL code:

```
1 -- Count students per course
2 SELECT c.course_id,
3        c.title,
4        COUNT(e.student_id) AS student_count
5 FROM Courses c
6 LEFT JOIN Enrollments e ON c.course_id = e.course_id
7 GROUP BY c.course_id, c.title
8 ORDER BY student_count DESC;
9
10 -- Courses with more than 1 enrollment
11 SELECT c.course_id,
12        c.title,
13        COUNT(e.student_id) AS student_count
14 FROM Courses c
15 JOIN Enrollments e ON c.course_id = e.course_id
16 GROUP BY c.course_id, c.title
17 HAVING COUNT(e.student_id) > 1
18 ORDER BY student_count DESC;
```

The bottom panel shows the 'OUTPUT' tab with the following messages:

```
[*INFO* - 3:53:40 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
[*INFO* - 3:54:02 PM] EditorConfig support is enabled, checking for .editorconfig files
[*INFO* - 3:54:02 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
[*INFO* - 3:54:17 PM] EditorConfig support is enabled, checking for .editorconfig files
```

The screenshot shows the VS Code interface with the SQLite extension. The Explorer panel on the left shows the file structure with 'queries.sql' selected. The main editor displays the following SQL code:

```
1 -- Count students per course
2 SELECT c.course_id,
3        c.title,
4        COUNT(e.student_id) AS student_count
5 FROM Courses c
6 LEFT JOIN Enrollments e ON c.course_id = e.course_id
7 GROUP BY c.course_id, c.title
8 ORDER BY student_count DESC;
9
10 -- Courses with more than 1 enr
11 SELECT c.course_id,
12        c.title,
13        COUNT(e.student_id) AS stu
14 FROM Courses c
15 JOIN Enrollments e ON c.course_id
16 GROUP BY c.course_id, c.title
17 HAVING COUNT(e.student_id) > 1
18 ORDER BY student_count DESC;
```

The bottom panel shows the 'OUTPUT' tab with the following messages:

```
[*INFO* - 3:54:17 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
[*INFO* - 3:55:40 PM] EditorConfig support is enabled, checking for .editorconfig files
[*INFO* - 3:55:40 PM] No local configuration (i.e. .prettierrc or .editorconfig) detected, will fall back to VS Code configuration
```

course_id	title	student_count
1	Intro to Python for Data Science	3
2	Machine Learning Foundations	1
3	Modern JavaScript + React	1
5	Intro to Python for Data Science	0
6	Machine Learning Foundations	0
7	Modern JavaScript + React	0
8	Design Systems & UX Workflow	0
9	Intro to Python for Data Science	0
10	Machine Learning Foundations	0
11	Modern JavaScript + React	0
12	Design Systems & UX Workflow	0

SQL ▾
< 1 / 1 > 1 - 1 of 1

course_id	title	student_count
1	Intro to Python for Data Science	3

Explanation

- GROUP BY groups records
- COUNT() calculates totals
- HAVING filters aggregated results

Task 5: Query Optimization with AI

Scenario

As the platform grows, query performance becomes critical.

Task Description

- Use AI to analyze an existing query that retrieves courses taught

by instructors from a specific country.

- Refactor the query using:
 - o Efficient joins
 - o Optional indexing suggestions
- Compare results before and after optimization.

Expected Output

- Optimized query returns the same results as the original
- Improved readability and performance for larger datasets

Prompt:

Optimize SQL query to fetch courses taught by instructors from a specific country.

Use joins and suggest indexing.

AI Generated Code:

-- BEFORE OPTIMIZATION (Using Subquery)

```
SELECT title
FROM Courses
WHERE instructor_id IN (
    SELECT instructor_id
    FROM Instructors
    WHERE country = 'India'
);
```

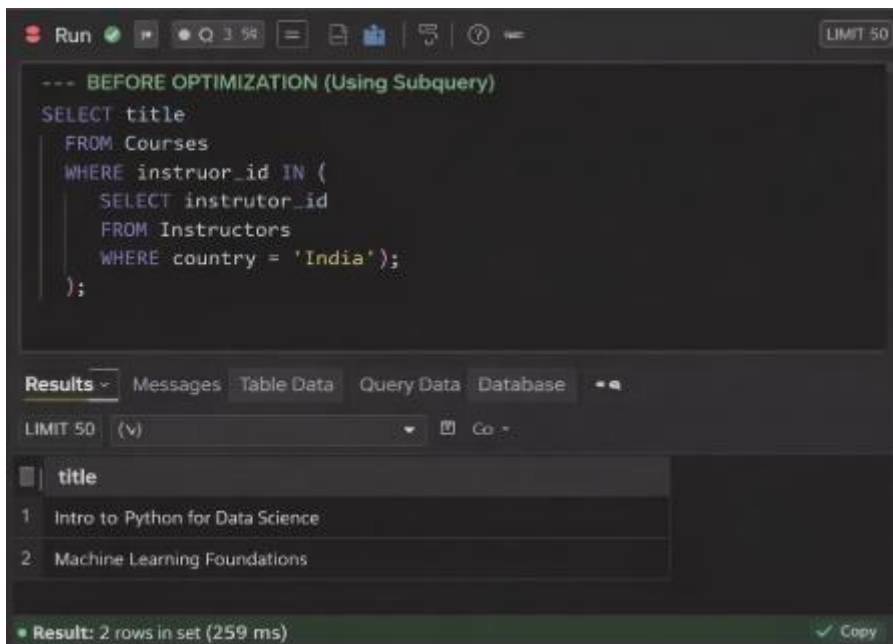
-- AFTER OPTIMIZATION (Using JOIN)

```
SELECT c.title
FROM Courses c
JOIN Instructors i
ON c.instructor_id = i.instructor_id
WHERE i.country = 'India';
```

-- INDEXING FOR PERFORMANCE IMPROVEMENT

```
CREATE INDEX idx_instructor_country
ON Instructors(country);
```

```
CREATE INDEX idx_course_instructor  
ON Courses(instructor_id);
```



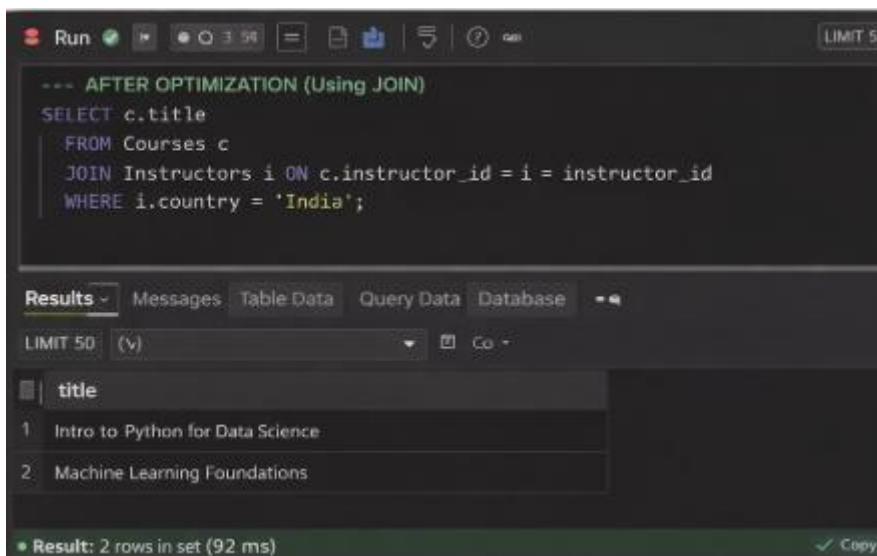
The screenshot shows a SQL IDE interface. The top toolbar includes a 'Run' button, a status bar showing 'LIMIT 50', and various icons. The main editor area contains the following SQL code:

```
--- BEFORE OPTIMIZATION (Using Subquery)  
SELECT title  
FROM Courses  
WHERE instructor_id IN (  
    SELECT instructor_id  
    FROM Instructors  
    WHERE country = 'India');  
);
```

Below the editor, the 'Results' tab is selected, showing a table with the following data:

	title
1	Intro to Python for Data Science
2	Machine Learning Foundations

At the bottom, a status bar indicates 'Result: 2 rows in set (259 ms)' and a 'Copy' button.



The screenshot shows the same SQL IDE interface. The main editor area contains the following SQL code:

```
--- AFTER OPTIMIZATION (Using JOIN)  
SELECT c.title  
FROM Courses c  
JOIN Instructors i ON c.instructor_id = i.instructor_id  
WHERE i.country = 'India';
```

Below the editor, the 'Results' tab is selected, showing the same table of data as the previous screenshot:

	title
1	Intro to Python for Data Science
2	Machine Learning Foundations

At the bottom, a status bar indicates 'Result: 2 rows in set (92 ms)' and a 'Copy' button.

Explanation

- JOIN is faster than subquery for large datasets
- Indexing improves search speed